Member 2 Briefing: CPU FP64 Reference Implementation

***Beginner foundation: you do not need to be a C++ programmer to explain the code flow.** Learn the purpose of the files and the movement of data first.*

Beginner foundation: how code is organized

What is a source file?

A source file contains human-written instructions. In this project, a .cpp file contains C++ instructions. A function is a named group of instructions that does one job. For example, read_matrix_market reads the input matrix.

A .hpp header is a shared description. It tells other source files which functions exist and what information they use. It is like a menu: it describes what can be ordered, while the .cpp file contains the kitchen work.

What is CPU FP64?

CPU means the normal main processor in a computer. FP64 means 64-bit floating-point numbers, called double in C++. A floating-point number stores decimal-like values such as 0.3; FP64 keeps roughly 16 useful decimal digits. The CPU FP64 version is the easiest implementation to follow first.

What are data and flow?

The main data objects are:

```
A : the input matrix
b : the known right-hand-side vector
P : sparse helper/preconditioner
x : current answer
r : residual, meaning b-A*x
d : correction found by GMRES
```

The code flow is simply the route these objects take from file to final answer:

```
text file -> A -> P -> x and r -> d -> improved x -> printed result
```

What is a class or object?

A class is a C++ blueprint for a kind of data. An object is an actual value made from that blueprint. CsrMatrix is the blueprint used to store a sparse matrix. Its object contains the matrix's nonzero values and functions such as multiply.

The remaining sections name the exact files that carry out each arrow in this flow. Read them slowly; you are learning architecture, not memorizing syntax.

How to read source code without getting lost

Ignore punctuation that you do not understand at first. Look for: the file name, function names, and the information entering/leaving each function. CsrMatrix::multiply(x) means "ask the matrix object to multiply by vector x." return means "give this result back." if means "continue only if this condition is true." A status is a named result such as success, invalid input, or iteration limit.

Code words decoded before they appear

Word - Beginner meaning

Parse - Read text and turn it into structured program data.

Canonical - Stored in one clean, predictable standard form.

API - Functions/data one file offers other files.

Options - Settings such as tolerances and iteration limits.

Result object - A structured return value containing answer, status, and diagnostics.

Diagnostic - An extra measurement that explains quality or failure.

Rank problem - The fitting equations lack enough independent information for the requested calculation.

Your role in one sentence

Explain how the CPU FP64 executable turns a Matrix Market file into a sparse solution using CSR storage, SPAI construction, GMRES, and iterative refinement.

Scope

Focus on the active CPU double-precision path. Ignore CUDA and FP32 unless someone asks for a comparison. The executable is cpu_fp64_ir.

The source flow is:

```
src/solver_main.cpp
  -> src/cpu.cpp
  -> src/ir.cpp
       -> src/spai.cpp
       -> src/gmres.cpp
            -> src/cpu.cpp (sparse matrix-vector products)
```

1. Program entry: src/solver_main.cpp

The main function accepts:

```
MATRIX.mtx [epsilon=0.3] [outercap=20] [innercap=0]
```

- MATRIX.mtx is a Matrix Market coordinate file.
- epsilon is the SPAI construction tolerance.
- outercap limits refinement corrections and must be at most 20.
- innercap=0 means GMRES may use up to n iterations; otherwise it must be between 1 and n.

The program reads the matrix, requires it to be nonempty and square, chooses

$$\beta = 8, \qquad \alpha = \lceil n/8 \rceil,$$

and constructs a unit-2-norm equal-component RHS:

```
b_i=1/sqrt n.
```

It calls spai_gmres_ir(matrix, rhs, options). It prints the precision, backend, matrix size, nonzero count, chosen tolerances, final status, outer steps, backward error, and solution entries. Exit code 0 means convergence; 1 means a construction or solver failure; 2 means invalid input.

2. Input and sparse storage: src/cpu.cpp

Matrix Market parsing

read_matrix_market accepts real or integer coordinate-format Matrix Market files, with general or symmetric storage. A coordinate file gives triples:

```
row column value
```

Matrix Market indices begin at 1; C++ arrays begin at 0, so the parser subtracts one. Symmetric off-diagonal entries are mirrored. Invalid dimensions, invalid indices, nonfinite values, and unsupported formats are rejected.

CSR format

The project stores matrices as compressed sparse row (CSR):

```
values         = nonzero stored values
column_indices = corresponding column for each value
row_offsets    = start/end position of each row in those arrays
```

For example,

```
begin{bmatrix}10&0&20&5&0-1&0&0end{bmatrix}
```

is stored as:

```
values         [10, 2, 5, -1]
column_indices [ 0, 2, 1,  0]
row_offsets    [ 0, 2, 3,  4]
```

The range for row i is row_offsets[i] up to but excluding row_offsets[i+1]. This avoids storing all the zeros.

Canonical construction

CsrMatrix::from_entries validates entries, stable-sorts them by row and column, sums duplicate locations in input order, removes exact resulting zeros, and creates the offsets. This gives a canonical sparse representation.

Sparse matrix-vector multiplication

CsrMatrix::multiply(x) computes y=Ax row by row:

$$y_i = \sum_{j \text{ stored in row } i} a_{ij} x_j.$$

It validates vector dimensions and finite values. This is the most frequently used primitive in the solver: SPAI diagnostics, GMRES operator application, residuals, and refinement all depend on it.

3. SPAI construction: src/spai.cpp

construct_spai(a, options) returns a SpaiResult. That result holds scaling, the scaled transpose, the intermediate inverse, the final preconditioner, and per-column diagnostics.

Scaling and orientation

For every original row, it verifies a nonzero diagonal and finds the maximum absolute entry. It computes a reciprocal scaling factor. It then builds B=A^T D, retaining sparse structure.

Column construction

For every column k, construct_column starts with the pattern {k}. It finds the shadow rows touched by that pattern. It forms a small dense reduced matrix and solves its least-squares fit with Householder QR.

The code records:

- the current sparsity pattern;
- QR coefficients;
- reduced and full residuals;
- fit and growth counts;
- candidate scores and selected candidates;
- a QR pivot-ratio diagnostic.

If the fit does not reach epsilon, it builds candidates from the shadow, scores them, selects at most beta whose

scores are no larger than the mean, and refits. It stops on success, exhausted growth budget, no candidates, a rank problem, or nonfinite numerical arithmetic.

When every column is complete, the code assembles inverse and the final preconditioner matrix P=N^T D.

4. GMRES correction solver: src/gmres.cpp

left_preconditioned_gmres(a, p, rhs, options) returns a GmresResult. During refinement, rhs is actually the current original residual r, and the initial guess is zero.

The operator is implemented as:

```
p.multiply(a.multiply(v))
```

which means P(Av). The code deliberately does not form a rounded dense or sparse P*A matrix.

GMRES validates dimensions and finite input, calculates the original initial residual, then calculates the preconditioned RHS Pr. It builds an Arnoldi Krylov basis with one modified Gram-Schmidt pass, stores the upper-Hessenberg matrix, applies Givens rotations, solves the small triangular system, and forms a candidate correction.

It verifies candidates by recomputing

$$\|Pr - PAd\|_2 / \|Pr\|_2 .$$

For FP64 the tolerance is 1e-8. The code has explicit status paths for exact initial solutions, zero preconditioned RHS, Arnoldi breakdown, iteration cap, nonconvergence, and nonfinite arithmetic.

5. Iterative refinement: src/ir.cpp

spai_gmres_ir validates its inputs, builds SPAI, and passes P to refine_with_preconditioner.

The refinement routine:

- 1. Sets initial.x = P*b.
- 2. Calculates residual and backward-error diagnostics.
- 3. If accurate, returns immediately.
- 4. Otherwise, runs at most max_outer_steps corrections.
- 5. Calls GMRES with the latest residual.
- 6. Adds the returned correction to x.
- 7. Recomputes diagnostics for the new x.
- 8. Accepts only if original-system backward error passes.

The diagnostic function computes residual infinity norm, residual 2-norm, normwise backward error, and componentwise backward error. Optional forward error is only available when a reference solution is supplied; the normal CLI does not have one.

Important implementation choices

- The reference CPU path is IEEE binary64 (double).
- GMRES is unrestarted and uses one MGS pass.
- The preconditioner is left-applied.
- The solver checks original-system backward error after every outer update.
- P is reused for all outer refinement corrections.
- The implementation uses named failure states rather than silently applying

a different numerical fallback.

Your demo narrative

"The executable parses a sparse Matrix Market input into CSR. CSR keeps only nonzero entries, so matrix-vector products are efficient. The solver builds a sparse approximate inverse preconditioner column by column using QR-based least-squares fits. It then starts from Pb. If that is not accurate enough, iterative refinement computes the residual and asks left-preconditioned GMRES for a correction. The update is accepted only when the original system meets the strict backward-error criterion."

Questions you should answer

6. Exact CPU FP64 function flow

This is the route to follow in a live code demonstration. Do not begin by reading every helper. Begin at the executable and descend one level at a time.

```
src/solver_main.cpp
  read command-line options
  read Matrix Market file
  create known test vector and right-hand side
  call cpu_spai_gmres_ir(...)

src/ir.cpp : cpu_spai_gmres_ir
  call construct_spai(A, options)
  call refinement routine with A, b, and P

src/ir.cpp : refinement loop
  sparse matrix-vector products from cpu.cpp
  call gmres_left_preconditioned(A, P, residual, ...)
  update solution
  evaluate stopping rule

src/gmres.cpp : gmres_left_preconditioned
  repeatedly apply A and P using CSR SpMV
  Arnoldi orthogonalization
  Givens rotations and small triangular solve
  explicit correction verification

solver_main.cpp
  independent final diagnostics
  formatted output
```

The output is not merely a vector. Result structures carry a status, iteration counts, histories, diagnostics, and the final solution. This is important because a vector produced after a breakdown must not be presented as a successfully verified answer.

7. Input path in slow motion

Matrix Market

A Matrix Market coordinate file is a text file. Its header declares properties such as real/integer values and general/symmetric structure. A size line gives the row count, column count, and number of listed entries. Each following line gives row column value using one-based indices.

The parser must do more than read numbers. It validates dimensions and indices, expands symmetry when the format says only one triangle is stored, converts one-based indices to C++ zero-based indices, combines duplicate entries, and creates a deterministic canonical sparse matrix. "Deterministic" means the same valid input produces the same ordered representation each run.

CSR with a concrete example

For

```
A=begin{bmatrix}10&0&20&3&04&0&5end{bmatrix},
```

CSR stores:

```
values  = [10, 2, 3, 4, 5]
columns = [ 0, 2, 1, 0, 2]
row_ptr = [ 0, 2, 3, 5]
```

row_ptr[0]=0 and row_ptr[1]=2 mean row 0 occupies positions 0 and 1 in the other arrays. Multiplying A*x walks only through those five stored nonzeros. This operation is called SpMV: sparse matrix-vector multiplication.

8. SPAI construction in code-level stages

construct_spai accepts the canonical CSR matrix and options. It prepares the scaled/oriented representation required by the column algorithm. Each output column maintains a current pattern and its coefficients.

At one pattern-growth step, the implementation identifies relevant rows, extracts a small dense reduced system, solves its least-squares problem with QR, measures the fitted residual, and either stops or scores shadow candidates. The final columns are assembled into a canonical sparse P.

Why create a small dense system inside a sparse algorithm? The complete matrix is large and sparse, but one SPAI column currently permits only a small set of unknown entries. Restricting the least-squares problem to that set makes a compact dense calculation practical.

Why QR? A least-squares problem asks for coefficients that minimize a norm even when there is no exact solution. QR is generally more numerically reliable than forming and solving the normal equations because the latter can square the conditioning difficulty.

9. GMRES in code-level stages

The CPU routine receives A, P, and the outer residual r.

- 1. Compute the preconditioned right-hand side z=P*r.
- 2. Handle the zero or already-solved case before allocating a large basis.
- 3. Normalize z to create the first Krylov basis vector.
- 4. Apply A and then P to the newest basis vector. This realizes the operator P*A without explicitly multiplying the sparse matrices together.
- 5. Orthogonalize the result against all earlier basis vectors. Dot products measure overlap; scaled vector additions remove it.
- 6. Store coefficients in the small Hessenberg matrix.
- 7. Apply previous and new Givens rotations to maintain a small triangular least-squares problem.
- 8. Check the estimated residual and the iteration cap.
- 9. Solve the small triangular system for coefficients y.
- 10. Form the correction d = d0 + V*y.
- 11. Reapply the actual sparse operators and verify the true preconditioned residual ratio.

The basis vectors contain full problem-sized vectors. The Hessenberg matrix is small relative to A, but grows with the number of iterations. Unrestarted GMRES retains the basis, which is why memory can become a practical limit.

10. Iterative refinement and stopping

The outer routine forms r=b-A*x in CPU memory. It computes

$$\eta = \frac{\|r\|_\infty}{\|A\|_\infty \|x\|_\infty + \|b\|_\infty}.$$

The infinity norm of a vector is its largest absolute entry. The infinity norm of a matrix is its largest absolute row sum. The denominator makes the measure meaningful across differently scaled problems.

If eta is acceptable, refinement stops successfully. Otherwise GMRES returns a correction d; the routine updates x=x+d and repeats until success, an explicit failure, or the outer step cap. The current FP64 whole-solution threshold is based on approximately 10 * 2^-53.

The CPU is therefore the clearest reference path: parsing, sparse operations, outer decisions, verification, and output all occur in ordinary host memory. It is valuable even when the GPU is faster because it provides an understandable comparison target.

11. How to trace one run yourself

Use this checklist while reading the source:

- 1. In solver_main.cpp, identify where A, b, and options are created.
- 2. Find the call returning the IR result. Write down the result type.
- 3. In ir.cpp, locate SPAI construction, initial guess, residual formation, GMRES call, update, and backward-error test.
- 4. In spai.cpp, follow one column rather than every column at once.
- 5. In gmres.cpp, mark the lines corresponding to operator application, orthogonalization, rotations, solution reconstruction, and explicit verification.
- 6. In cpu.cpp, locate Matrix Market parsing, CSR construction, and CSR SpMV.
- 7. Return to the driver and match every printed field to the structure member it reports.

This route turns a large codebase into one nested story: driver, outer solver, correction solver, preconditioner builder, and sparse primitives.

12. Failure is part of the interface

Possible unsuccessful outcomes include invalid input, a singular or rank-deficient reduced QR problem, non-finite arithmetic, GMRES breakdown, failed correction verification, and reaching an iteration limit. A robust solver exposes these states instead of silently returning a vector. During a presentation, distinguish "the program ran" from "the solver returned a verified-success status."

13. Every CPU file involved

- include/bspai/cpu.hpp: declares Entry, CsrMatrix, accessors, multiplication, transpose, and Matrix Market input.
- src/cpu.cpp: implements parsing, canonical CSR creation, SpMV, and transpose.
- include/bspai/spai.hpp: declares SPAI options, statuses, histories, results, scoring, and constructor API.
- src/spai.cpp: implements scaling, QR helpers, column fitting, score selection, pattern growth, and assembly.
- include/bspai/gmres.hpp: declares GMRES options, statuses, iteration/result records, operator application, and solver API.
- src/gmres.cpp: implements vector helpers, residual verification, Arnoldi, Givens rotations, triangular

recovery, and termination.

- include/bspai/ir.hpp: declares refinement options, diagnostics, histories, status, and top-level APIs.
- src/ir.cpp: implements diagnostics, fixed-preconditioner refinement, and complete SPAI-GMRES-IR.
- src/solver_main.cpp: common CLI driver used by the solver variants.
- cmake/CpuTargets.cmake: tells CMake which files form the CPU libraries, tests, and executables.
- tests/test_cpu.cpp, test_spai.cpp, test_gmres.cpp, test_ir.cpp: component and algorithm validation.

14. CPU function and parameter reference

Sparse foundation

CsrMatrix::from_entries(rows, cols, entries) takes dimensions and (row,column,value) triplets. It validates, sorts, combines duplicates, drops exact zeros, and returns canonical CSR.

CsrMatrix::multiply(x) takes a vector of length cols() and returns a vector of length rows(). Dimension or non-finite problems are rejected.

CsrMatrix::transpose() takes no parameter and returns a new canonical CSR matrix with dimensions exchanged.

read_matrix_market(input) takes an input stream positioned at a Matrix Market document and returns a CsrMatrix; malformed headers, unsupported forms, invalid indices, or inconsistent counts throw an exception.

SPAI

evaluate_spai_score(residual_squared,dot,denominator_squared) returns ScoreResult {status, score, clamped}. It protects zero denominators and small negative roundoff artifacts.

construct_spai(A,options) takes SpaiOptions {epsilon=0.3, alpha=5, beta=8} by default. It returns SpaiResult. epsilon is the column residual target; alpha is the growth-step allowance; beta limits selected candidates per growth.

Important internal functions include at(A,row,col) for sparse lookup, norm2(v), dense QR helpers, and construct_column(B,k,options). The last returns SpaiColumn with coefficients, pattern, residuals, counts, status, and complete fit history.

GMRES

apply_left_preconditioned(A,P,v) returns P*(A*v) using two calls to CSR multiply.

left_preconditioned_gmres(A,P,rhs,options,initial_guess={}) takes GmresOptions {tolerance=1e-8,max_iterations=0}. A zero cap means use matrix dimension n; explicit caps must be in 1..n. It returns GmresResult, whose solution is the correction vector.

Internal helpers compute safe finite arithmetic, vector subtraction, residual checks, back substitution, candidate formation, and history storage. The outer function owns the basis, unrotated Hessenberg matrix, rotated working matrix, cosine/sine arrays, and transformed right-hand side g.

Iterative refinement

evaluate_ir_diagnostics(A,rhs,x,reference={}) returns residual vectors/norms, normwise and componentwise backward errors, and optional forward error.

refine_with_preconditioner(A,P,rhs,options,reference={}) returns a complete IrResult but does not construct P.

spai_gmres_ir(A,rhs,options,reference={}) first calls construct_spai, then refinement. IrOptions nests SpaiOptions, GmresOptions, and max_outer_steps=20.

15. Function-by-function execution of one CPU solve

- 1. main checks argc, parses numeric caps with parse_count, opens the file, and calls read_matrix_market.
- 2. read_matrix_market uses next_data_line, parse_size, and require_end, expands symmetry, and calls from_entries.
- 3. The driver checks square/nonempty input, constructs the normalized all-ones rhs, fills options, starts timing, and calls spai_gmres_ir.

- 4. spai_gmres_ir initializes IrResult, calls construct_spai, stops on construction failure, otherwise forwards P to refine_with_preconditioner.
- 5. construct_spai derives scaling and the scaled transpose B, invokes construct_column for k=0..n-1, and assembles inverse and preconditioner.
- 6. refine_with_preconditioner computes an initial preconditioner-based guess, calls evaluate_ir_diagnostics, and enters the serial outer loop.
- 7. If the backward-error test fails, it calls left_preconditioned_gmres with the current residual.
- 8. GMRES applies P*A, orthogonalizes, updates rotations, triggers verification, reconstructs d, and returns its status/history.
- 9. IR records the inner result and correction RHS, checks success and finite updates, performs x=x+d, recalculates diagnostics, and repeats.
- 10. The driver prints the nested result and returns success/failure to the operating system.

Every CPU loop described here is serial in this baseline implementation. That makes it the clearest reference for comparing CUDA parallelization without confusing a different mathematical method with an acceleration technique.

What is CSR and why use it? It stores only nonzeros and makes sparse matrix-vector products efficient.

What does cpu.cpp do? Matrix parsing, canonical CSR creation, sparse multiply, and transpose. It is infrastructure rather than the main algorithm.

Why not form P*A? Applying P(Av) preserves the intended sparse operation and avoids an extra materialized matrix.

What happens if SPAI cannot meet its target? The top-level solve reports a construction failure and does not treat a partial preconditioner as success.

What is the difference between a GMRES residual and final backward error? GMRES uses a preconditioned correction residual; final acceptance checks the original Ax=b system.